

Function Design

เกมปริศนาล่นบนกระดานสี่เหลี่ยมขนาด 8x8 ช่อง เป้าหมายคือวางชิ้นส่วนรูปทรงต่าง ๆ ลงบนกระดานให้เต็มเต็มแถวหรือคอลัมน์ใด ๆ จนครบ เพื่อ "ระเบิด" (ล้าง) แถว/คอลัมน์นั้นทิ้งแล้วได้คะแนน โดยไม่มีเวลาจำกัด แต่เกมจะจบเมื่อกระดานแน่นอนจนไม่มีที่ให้วางชิ้นส่วนที่เหลือในมือได้อีก

ฟังก์ชัน / เมธอด	หน้าที่การทำงาน	Parameters	Return
1. ฟังก์ชันทั่วไป / Global Functions			
<code>draw_square</code> (x, y, size, fill_color, stroke_color, corner_weight):	วาดสี่เหลี่ยมจัตุรัสหนึ่งช่องบนจอ โดยวาดเส้นแนวนอนเรียงกันทีละพิกเซล เพื่อเติมสีพื้น (fill) จากนั้นวาดเส้นขอบทั้ง 4 ด้าน ทับด้านบนด้วยสีและความหนาที่กำหนด	x, y— ตำแหน่งมุมซ้ายบนของช่อง size— ความยาวด้านของสี่เหลี่ยม (พิกเซล) fill_color— สีพื้น (R,G,B) stroke_color — สีเส้นขอบ (R,G,B) corner_weight— ความหนาเส้นขอบ	ไม่คืนค่า (None)
<code>spawn_hand()</code> :	สุ่มชิ้นส่วนใหม่ 3 ชิ้นจากลิสต์ SHAPE_TEMPLATES มาใส่ในมือผู้เล่น (global hand) พร้อมคำนวณตำแหน่ง x, y ที่จะวางแสดงผลแต่ละชิ้น โดยแบ่งความกว้างจอออกเป็น 3 ช่อง	ไม่มีพารามิเตอร์	ไม่คืนค่า (None)
<code>is_hand_empty()</code> :	ตรวจสอบว่าชิ้นส่วนทุกช่องในมือ (hand) ถูกใช้ไปหมดแล้วหรือยัง โดยวนลูปเช็คค่าทุกช่องมีค่าเป็น 0 หรือไม่	ไม่มีพารามิเตอร์	คืนค่า bool True หากทุกช่องว่าง (ใช้ครบแล้ว)

ฟังก์ชัน / เมธอด	หน้าที่การทำงาน	Parameters	Return
			False หากยังมีชิ้นส่วนเหลืออยู่ อย่างน้อย 1 ชิ้น
check_game_over():	ตรวจสอบว่าเกมจบหรือยัง โดยรวบรวมชิ้นส่วนที่ไม่ซ้ำกันในมือ (unique_pieces) แล้ววนตรวจสอบทุกช่อง (row, column) บนกระดานว่ามีช่องใดที่ยังวางชิ้นส่วนนั้นได้หรือไม่ (เรียก board.can_place) หากไม่มีชิ้นส่วนใดวางได้เลยแม้แต่ช่องเดียว ถือว่าเกมจบ	ไม่มีพารามิเตอร์	คืนค่า bool True = เกมจบ (ไม่มีที่ว่างเหลือ), False = ยังเล่นต่อได้
setup():	ฟังก์ชันเริ่มต้น เรียกทำงานครั้งเดียวตอนเริ่มโปรแกรม (และถูกเรียกซ้ำเมื่อคลิกเพื่อรีเซ็ตาร์ทหลังแพ้) กำหนดขนาดหน้าต่าง (size), สร้าง Board ใหม่, รีเซ็ตคะแนนและสถานะ game_over, จากนั้นสุ่มชิ้นส่วนแรกด้วย spawn_hand()	ไม่มีพารามิเตอร์	ไม่คืนค่า (None)
draw():	วาดภาพทั้งหมด: วาดพื้นหลัง, วาดกระดาน (board.draw), วาดชิ้นส่วนในมือที่ยังไม่ถูกลาก, วาดชิ้นส่วนที่กำลังถูกลาก/ลากอยู่บนสุด, แสดงข้อความ "YOU LOSE" เมื่อ game_over เป็นจริง และแสดงคะแนนปัจจุบัน	ไม่มีพารามิเตอร์	ไม่คืนค่า (None)

ฟังก์ชัน / เมธอด	หน้าที่การทำงาน	Parameters	Return
mousePressed():	เมื่อผู้เล่นคลิกเมาส์ ถ้า game_over เป็นจริงจะเรียก setup() เพื่อเริ่มเกมใหม่ที่ มิฉะนั้นจะวนหาชิ้นส่วนในมือที่ตำแหน่งเมาส์คลิกโดน (ผ่าน contains_point) หากพบจะตั้งให้เป็น selected_piece, บันทึก selected_index และเริ่มสถานะลาก (is_dragging = True) พร้อมคำนวณ drag offset	ไม่มีพารามิเตอร์	ไม่คืนค่า (None)
mouseDragged():	ทำงานขณะผู้เล่นลากเมาส์ อัปเดตตำแหน่ง x, y ของ selected_piece ให้เคลื่อนตามตำแหน่งเมาส์ปัจจุบัน (หักลบด้วย drag_offset ที่บันทึกไว้ตอนคลิก)	ไม่มีพารามิเตอร์	ไม่คืนค่า (None)
mouseReleased():	เมื่อผู้เล่นปล่อยเมาส์ คำนวณช่องเป้าหมาย (target_r, target_c) บนกระดานจากตำแหน่งปัจจุบันของชิ้นส่วน ตรวจสอบด้วย board.can_place ว่าวางได้หรือไม่: ถ้าวางได้ จะเรียก board.place, บวกคะแนนจาก (จำนวนบล็อก x 10) รวมกับคะแนนจาก board.clear_lines(), ล้างชิ้นส่วนออกจากมือ, เติมนิหม์ถ้าหมด และตรวจ check_game_over() ถ้าวางไม่ได้ จะเรียก selected_piece.reset_pos() เพื่อคืนตำแหน่งเดิม สุดท้ายเคลียร์ selected_piece และ selected_index	ไม่มีพารามิเตอร์	ไม่คืนค่า (None)

ฟังก์ชัน / เมธอด	หน้าที่การทำงาน	Parameters	Return
2. Class Board :— จัดการกระดานเกม			
__init__ (self, size, cell_size, origin_x, origin_y):	Constructor ของคลาส Board สร้างกระดานเกมขนาด size x size โดยสร้าง grid แบบ 2 มิติ (list ซ้อน list) เริ่มต้นทุกช่องเป็น 0 (ว่าง) และเก็บค่าขนาดช่อง/ตำแหน่งเริ่มวาดไว้ใช้อ้างอิงตอนวาดและคำนวณตำแหน่ง	self size — จำนวนช่องต่อด้าน (เช่น 8) cell_size — ขนาดพิกเซลต่อ 1 ช่อง origin_x, origin_y — พิกัดมุมซ้ายบนของกระดานบนจอ	ไม่คืนค่า (None)
draw(self):	วาดกระดานทั้งหมดโดยวนลูปทุกช่องใน self.grid ถ้าค่าช่องเป็น 0 (ว่าง) จะวาดพื้นขาวขอบดำ แล้วเรียก draw_square เพื่อวาดแต่ละช่องตามตำแหน่งจริงบนจอ	self	ไม่คืนค่า (None)
can_place(self, piece, target_r, target_c):	ตรวจสอบว่าสามารถวาง piece ลงบนกระดานโดยให้บล็อกแรกอยู่ที่ (target_r, target_c) ได้หรือไม่ โดยวนตรวจสอบทุกบล็อกของ piece ต้องอยู่ในขอบเขตกระดาน (ไม่ติดขอบ ไม่เกิน size) และช่องนั้นต้องว่าง (ค่าเป็น 0) หากมีบล็อกใดติดเงื่อนไขจะคืนค่า False ทันที	self piece — ชิ้นส่วนที่ต้องการตรวจสอบ target_r — แถวเป้าหมายที่จะวางบล็อกจุดอ้างอิง target_c — คอลัมน์เป้าหมายที่จะวางบล็อกจุดอ้างอิง	คืนค่า bool True หากวางได้ทุกบล็อก, False หากมีบล็อกใดชนขอบเขตหรือทับช่องที่มีอยู่แล้ว
place(self, piece, target_r, target_c):	วางชิ้นส่วน piece ลงบน self.grid จริง	self piece — ชิ้นส่วนที่จะวาง target_r — แถวเป้าหมาย target_c — คอลัมน์เป้าหมาย	ไม่คืนค่า (None)

ฟังก์ชัน / เมธอด	หน้าที่การทำงาน	Parameters	Return
<code>clear_lines(self):</code>	ตรวจสอบทุกแถวและทุกคอลัมน์ว่าเต็ม (ไม่มีช่องใดเป็น 0) หรือไม่ เก็บรายการแถว/คอลัมน์ที่เต็มไว้ใน <code>rows_to_clear</code> และ <code>cols_to_clear</code> จากนั้นล้างค่าทุกช่องในแถว/คอลัมน์ที่เต็ม ให้กลับเป็น 0 แล้วคำนวณคะแนนจากจำนวนแถว+คอลัมน์ที่ล้างได้ทั้งหมด	<code>self</code>	int คะแนนที่ได้ = (จำนวนแถวที่ล้าง + จำนวนคอลัมน์ที่ล้าง) × 100
3. Class Piece : — จัดการชิ้นส่วนเกม			
<code>__init__(self, blocks, color_idx, anchor_x, anchor_y):</code>	Constructor ของคลาส Piece เก็บรูปทรงของชิ้นส่วน (blocks), ดัชนีสี, ตำแหน่ง anchor ที่ชิ้นส่วนจะกลับไปอยู่เมื่อวางไม่สำเร็จ และตั้งค่าเริ่มต้นของสถานะการลาก (<code>is_dragging</code>, <code>drag_offset</code>) รวมถึงขนาดช่องเล็กตอนแสดงในมือ (<code>mini_cell</code>)	<code>self</code> <code>blocks</code> — รูปทรงของชิ้นส่วน (ตำแหน่งสัมพัทธ์ของแต่ละบล็อก) <code>color_idx</code>— ดัชนีสีใน PALETTE <code>anchor_x</code>, <code>anchor_y</code> — ตำแหน่งเริ่มต้น/ตำแหน่งกลับเมื่อวางไม่สำเร็จ	ไม่คืนค่า (None)
<code>draw(self):</code>	วาดชิ้นส่วนบนจอ โดยวนวาดทุกบล็อกด้วย <code>draw_square</code> ใช้ <code>mini_cell</code> (ช่องเล็ก) ตามปกติ แต่ถ้ากำลังลากอยู่ (<code>is_dragging</code>) จะใช้ <code>CELL_SIZE</code> (ขนาดเท่ากระดาน) แทน	<code>self</code>	ไม่คืนค่า (None)

ฟังก์ชัน / เมธอด	หน้าที่การทำงาน	Parameters	Return
contains_point (self, px, py):	ตรวจสอบว่าจุด (px, py) เช่น ตำแหน่งเมาส์ อยู่ในขอบเขตของบล็อกใดบล็อกหนึ่งของ ชิ้นส่วนหรือไม่(ใช้สำหรับตรวจจับว่าเมาส์คลิกโดนชิ้นส่วนนี้หรือไม่ใน mousePressed)	self px, py— พิกัดจุดที่ต้องการตรวจสอบ (mouseX, mouseY)	คืนค่า bool True หากจุดอยู่ในบล็อกใด บล็อกหนึ่งของชิ้นส่วน, False หากไม่อยู่ในบล็อกใดเลย
reset_pos(self):	รีเซ็ตตำแหน่งของชิ้นส่วนกลับไป anchor_x, anchor_y (ตำแหน่งเดิมในมือผู้เล่น) และยกเลิกสถานะการลาก เรียกใช้เมื่อผู้เล่นปล่อยชิ้นส่วนในตำแหน่งที่วางไม่ได้	self	ไม่คืนค่า (None)

- คะแนนรวมต่อการวางหนึ่งครั้งคำนวณจาก 2 ส่วน คือ (จำนวนบล็อกของชิ้นส่วน $\times 10$) จากการวาง บวกกับคะแนนจาก board.clear_lines() ซึ่งคือ (จำนวนแถว+คอลัมน์ที่ล้างได้) $\times 100$